
Protein Circuit Topology Plugin

User Manual & Documentation

circuittopology@gmail.com

Abstract

This is the official documentation file for the PyMOL Protein Circuit Topology plugin. The plugin wraps most of the functionality of the ProteinCT framework [1] and exposes it through a PyMOL GUI. For questions, suggestions, or bug reports, please contact us via the email address above. The latest version is available at the [GitHub repository](#); the release described in the accompanying SoftwareX article is tag `v0.0.4`.

Circuit Topology in one paragraph. Circuit Topology (CT) [2] describes how pairs of residue–residue contacts — within a chain and, for multi-chain objects, between chains — are arranged along the sequence. A *contact pair* is a pair of residues in spatial proximity; a *relation* is the arrangement of two contact pairs. The plugin exposes the four relation types also returned by ProteinCT’s `local_ct`: **Series (S)** — sequentially arranged, non-overlapping contact pairs; **Parallel (P)** — one contact pair nested inside the other; **Inverse parallel (IP)** — inverse nesting; and **Cross (X)** — interleaved contact pairs. A relation is a statement about the order of the four endpoints along the sequence; in particular a Cross relation does not by itself imply a physical crossing, threading or knot. A *contact map* is the set of residue pairs within a distance cutoff; the plugin renders it as a residue $\times$ residue matrix or as an arc (“circuit”) diagram above the sequence. In a *relations matrix*, every contact-pair cell is coloured by its relation type. The exact definitions the plugin implements — which atoms are included, how contact pairs are detected, how multi-chain objects are treated, and two known limitations inherited from ProteinCT (insertion codes and chain breaks) — are given in Section 3.

Contents

1	Installation Guide	3
1.1	Downloading and Installing PyMOL	3
1.2	Mandatory Dependencies	3
1.3	Plugin Activation Steps	3
1.4	What the Automatic Installer Does	5
2	User Guide	6
2.1	Local Circuit Topology (LCT) Protocol	6
2.2	Single-File Analysis Protocol	11
2.2.1	Visualizing Relation Types on the 3D Structure in PyMOL	13
2.2.2	CT Folding Score	16
2.3	Multi-File Analysis Protocol	17
3	Methods specification	20
3.1	What is analysed	20
3.2	Contact pairs	21
3.3	Relations and relation types	21
3.4	Derived quantities	23
3.5	Batch and trajectory processing	24
3.6	Validation	25

1 Installation Guide

1.1 Downloading and Installing PyMOL

Download the PyMOL 3.1.6.1 installer for your platform (Windows / Linux / macOS) from <https://www.pymol.org/>. Previous versions are available at:

<https://storage.googleapis.com/pymol-storage/installers/index.html>

During setup, install PyMOL for the current user only by selecting **Just Me (recommended)**. This ensures the plugin’s automatic dependency installer has the necessary write permissions. If PyMOL is installed for all users, the installer needs administrator or root privileges; the plugin detects this, explains it, and prints the command to run yourself.

When PyMOL prompts you to register file extensions, select **Register Recommended**. The plugin works with `.pdb` and `.cif` structures and with `.xtc` trajectories.

1.2 Mandatory Dependencies

Table 1: Required Python and PyMOL versions. These are the versions the released outputs were produced with.

Dependency	Version	Dependency	Version
Python	3.10.18	NumPy	1.26.4
PyQt	5.15.11	Pandas	2.3.3
Qt-Main	5.15.15	Matplotlib	3.9.1
PyQt5-sip	12.17.0	PyMOL	3.1.6.1
Biopython	1.85	PyMOL-Bundle	3.1.6.1

Only four of these are the plugin’s own requirements — NumPy, Pandas, Matplotlib and Biopython — and they are declared in `requirements.yml`. Everything else ships with the PyMOL bundle. On a stock PyMOL 3.1.6.1 installation only `pandas` and `matplotlib` are actually missing, so those are the only two packages the automatic installer normally has to fetch.

1.3 Plugin Activation Steps

1. Download the latest plugin release `.zip` directly from GitHub:

[protein-circuit-topology-pymol-plugin.zip](#)

For a specific tagged version, browse the [Releases page](#); the version described in the SoftwareX article is [v0.0.4](#).

2. Open PyMOL and navigate to **Plugin** → **Plugin Manager**.

3. Switch to the **Install New Plugin** tab and click **Choose file...**, then select the .zip you just downloaded.

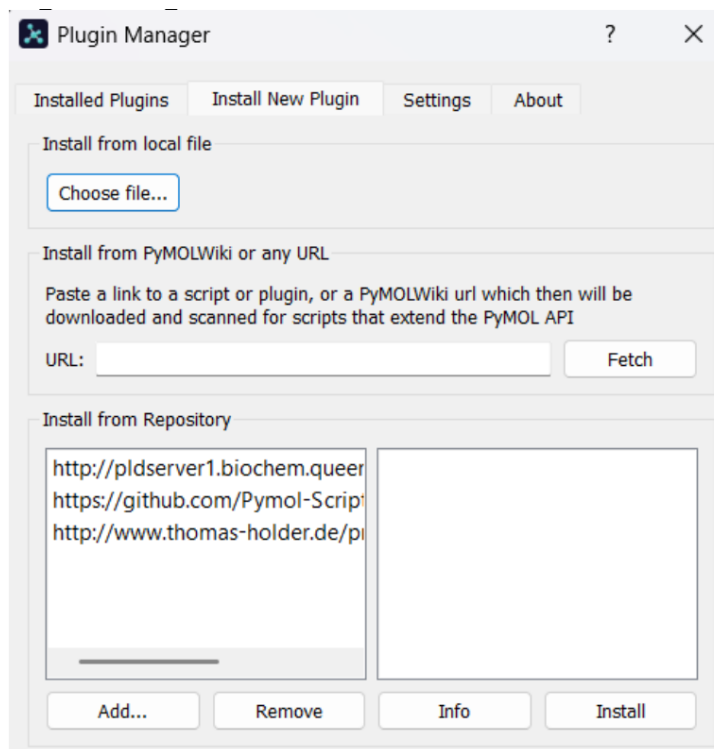


Figure 1: PyMOL's Plugin Manager with the **Install New Plugin** tab open. Use the **Choose file...** button under *Install from local file* to select the downloaded .zip.

4. Wait for the plugin to install and configure its dependencies. After installation, launch the plugin from **Plugin** → **Protein Circuit Topology Plugin**.

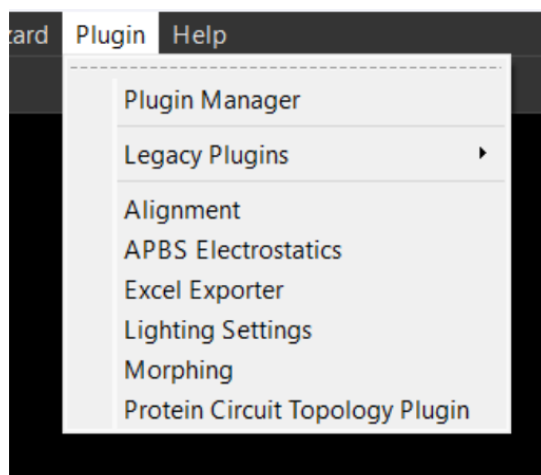


Figure 2: The plugin appears under the PyMOL **Plugin** menu after installation.

Important Installation Notes

- **Initial setup:** the first time the plugin is loaded, PyMOL may need extra time to download and configure the missing dependencies. Progress is printed to the PyMOL command line.
- **Restart prompt:** if the packages install correctly but Python cannot import them in the already-running session, the plugin says so and asks you to restart PyMOL. Nothing is lost — reopen PyMOL and the plugin is ready.
- **Nothing is replaced:** the installer only *adds* packages. PyMOL and Python are pinned to the versions you already have, so the solver cannot swap them out while resolving the new dependencies.

1.4 What the Automatic Installer Does

The installer works entirely through the conda environment that PyMOL itself runs in. It never creates a new environment and never touches any other environment on the system. In outline it

1. locates the conda belonging to the running PyMOL (CONDA_EXE first, then the usual paths under `sys.prefix`, then `conda` on `PATH`);
2. works out which of the four requirements are missing; and
3. installs only those, with the interpreter and PyMOL pinned:

```
conda install --yes --freeze-installed --prefix <PyMOL's sys.prefix>
python==<installed> pymol==<installed>
conda-forge::<each missing package>
```

The pins matter because they stop conda from satisfying the new dependencies by replacing PyMOL itself.

Manual Installation

If you would rather install the dependencies yourself, run the equivalent command before loading the plugin. Find the prefix and the versions to pin with

```
python -c "import sys, pymol; print(sys.prefix,
sys.version.split()[0], pymol.cmd.get_version()[0])"
```

and then

```
conda install --yes --freeze-installed --prefix <prefix>
python==<version> pymol==<version>
"conda-forge::numpy">=1.23,<2" "conda-forge::pandas">=2.0,<3"
"conda-forge::matplotlib">=3.7,<4" "conda-forge::biopython">=1.80,<2"
```

2 User Guide

The GUI opens as a single window titled *Circuit Topology Tool* with three tabs: **Local Circuit Topology**, **Single-File Analysis** and **Multi-File Analysis**. Small grey *i* buttons throughout the interface explain individual controls on hover or click.

Non-polymer atoms are handled for you

Circuit Topology analyses polymer atoms only. Waters, ions, ligands and other non-polymer atoms are excluded automatically at analysis time, and nucleic-acid and UNK residues are dropped when the chain is parsed. Nothing is removed from your loaded object, and no cleanup step is required before running an analysis. When you pick an object, the plugin prints to the PyMOL command line exactly what it will ignore, for example:

```
Excluded 118 non-polymer atoms (58 HOH, 2 ZN) from analysis; 1bcs is unchanged.
```

2.1 Local Circuit Topology (LCT) Protocol

The Local Circuit Topology tab analyses the topological relationships of one targeted residue against the rest of the structure.

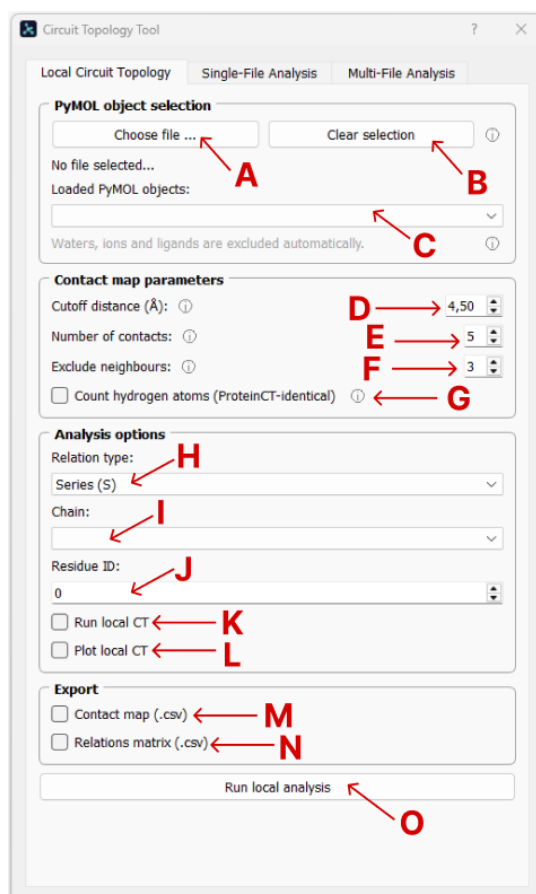


Figure 3: The Local Circuit Topology tab, with the controls used in this protocol labelled A–N: object selection (A–C), contact-map parameters (D) cutoff distance, (E) number of contacts, (F) exclude neighbours (the *Count hydrogen atoms* checkbox sits below (F)); analysis options (G) relation type, (H) chain, (I) residue ID, (J) run local CT, (K) plot local CT; exports (L) contact map, (M) relations matrix; and the run button (N).

To narrow the analysis to a specific region, select a chain (H) and a residue ID (I), then choose the relation type (G) of interest. With **Run local CT** (J) ticked the plugin computes, for every contact pair that begins or ends at the selected residue, how it relates topologically to every other contact pair in the chain. Results can be exported as .csv files via (L) and (M).

LCT Step-by-Step Guide

1. **Load a structure:** click **Choose file ...** (A) to load a .pdb or .cif file, or pick an already-loaded PyMOL object from the **Loaded PyMOL objects** dropdown (C). **Clear selection** (B) unloads the current file or object from PyMOL again.
2. **Define contact parameters (D–F):**
 - **Cutoff distance (Å) (D)** — maximum atom–atom distance counted as a contact. Range 0.1–20.0, default 4.5.
 - **Number of contacts (E)** — minimum number of atom–atom contacts required before a residue–residue pair counts as a contact. Range 5–45, default 5.

- **Exclude neighbours (F)** — discards trivial contacts between residues close in sequence. Range 1–10, default 3.
 - **Count hydrogen atoms (ProteinCT-identical)** — off by default, so only heavy atoms enter the atom–atom criterion and hydrogen-bearing files (NMR entries, MD frames) give the same contact pairs as their heavy-atom counterparts. Tick it to count every atom present, exactly as the reference ProteinCT implementation does (Section 3.1).
3. **Target a region:** pick the **Chain (H)** and the **Residue ID (I)**. The residue spinner is bounded by the residues actually present in the chosen chain, and it is repopulated whenever you change object or chain.
 4. **Choose Relation Type (G):** **Series (S)**, **Parallel (P)**, **Inverse parallel (IP)** or **Cross (X)**.
 5. **Output options:**
 - Tick **Run local CT (J)** to compute the per-residue relation-type counts, which are printed to the PyMOL command line.
 - Tick **Plot local CT (K)** to draw the plots shown below.
 - Tick **Contact map (.csv) (L)** or **Relations matrix (.csv) (M)**. Ticking either one reveals a **Choose output directory ...** button below the checkboxes; pick a destination before running.
 6. **Run:** click **Run local analysis (N)**.

When nothing is highlighted

Not every combination of chain, residue and relation type has anything to show. If the selected residue takes part in contact pairs but none of them stand in the requested relation, the plugin says so on the command line, and suggests trying another relation type or residue. If the residue takes part in no contact pairs at all, that is reported too — a common outcome when the cutoff distance is small or **Exclude neighbours** is large.

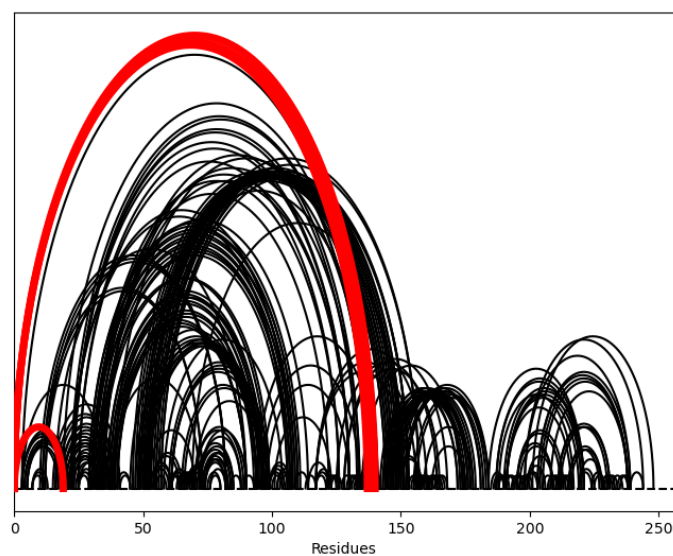


Figure 4: Local CT plot, step 1: contact pairs that begin or end at the selected residue are highlighted in red over the background contact map.

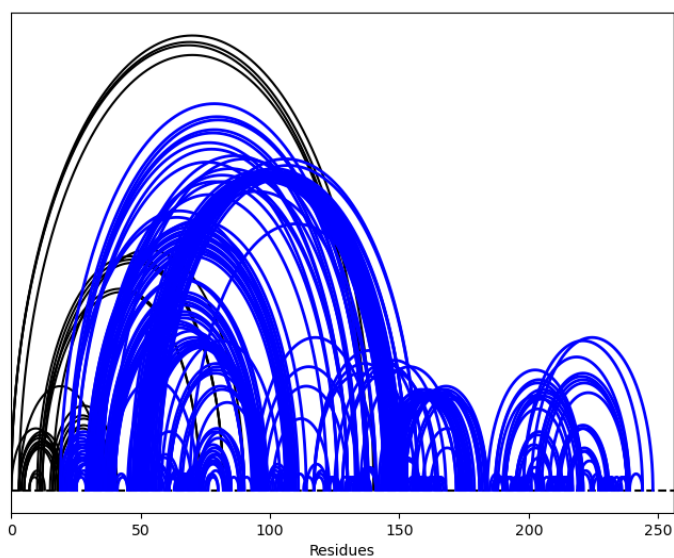


Figure 5: Local CT plot, step 2: contact pairs whose relation to the red ones is the relation type selected in dropdown (G) are drawn in blue.

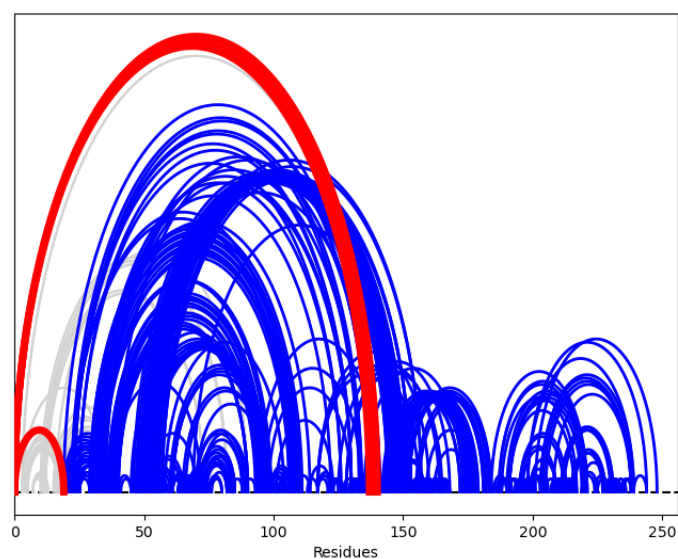


Figure 6: Combined local CT plot showing both the red (residue-anchored) and blue (relation-selected) curves, allowing visual identification of local topological relationships around the selected residue.

2.2 Single-File Analysis Protocol

The Single-File Analysis tab gives a global topological overview of one structure: a relations matrix, the corresponding circuit (arc) representation, an optional scalar **Folding Score** per chain, and direct recolouring of the structure in the PyMOL viewer.

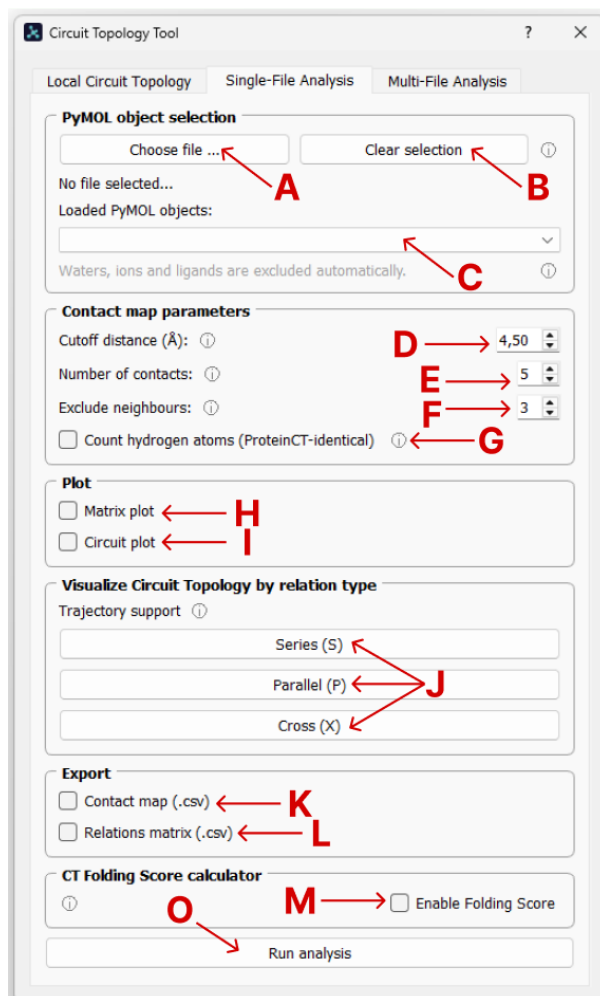


Figure 7: The Single-File Analysis tab, labelled A–M: object selection (A–C), contact-map parameters (D–F) (the *Count hydrogen atoms* checkbox sits below (F)), plot toggles (G) matrix and (H) circuit, the *Visualize Circuit Topology by relation type* buttons (I) (Series, Parallel, Cross), export checkboxes (J–K), the Folding Score toggle (L), and the run button (M).

The **Matrix plot** (G) renders the relations matrix, in which each cell shows the relation type between two contact pairs. For *single-chain* structures the matrix uses the standard S / P / X types together with their inverse and concerted variants. For *multi-chain* structures the whole model is analysed at once: inter-chain contact pairs are included, S, P and X are assigned only to two intra-chain contact pairs of the same chain, and pairs involving an inter-chain contact pair fall into the Independent (I), Tandem (T) or Loop (L) types visible in the legend of Figure 9 and defined in Section 3.3. The **Circuit plot** (H) draws every contact pair as an arc above the residue index; for multi-chain inputs each chain is rendered in a different colour (Figure 8).

Single-File Analysis Guide

1. **Load the protein (A–C):** load a `.pdb` or `.cif` file, or pick an existing PyMOL object from the dropdown.
2. **Define contact parameters (D–F):** cutoff distance, number of contacts and excluded neighbours — the same parameters, ranges and defaults as on the LCT tab, plus the **Count hydrogen atoms (ProteinCT-identical)** checkbox (off by default, see Section 3.1).
3. **Select plots:** tick **Matrix plot (G)** for the relations matrix and **Circuit plot (H)** for the arc diagram.
4. **(Optional) Folding Score:** tick **Enable Folding Score (L)** to compute a per-chain CT Folding Score (Section 2.2.2).
5. **Export data:** tick **Contact map (.csv) (J)** and/or **Relations matrix (.csv) (K)**, then choose an output directory in the widgets that appear.
6. **Run:** click **Run analysis (M)**.

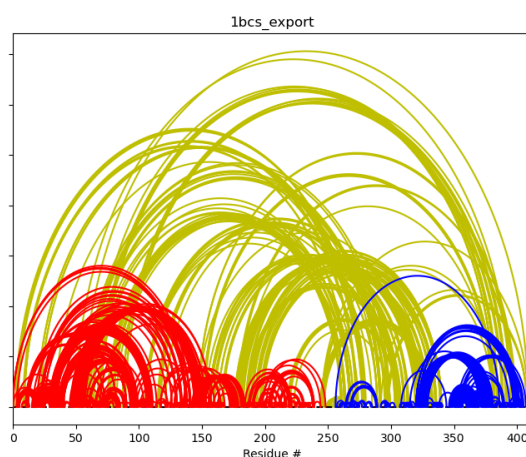


Figure 8: Circuit (arc) plot produced by the Single-File Analysis tab for the multi-chain object `1bcs_export`. Each chain is drawn in a distinct colour; arcs spanning chains are inter-chain contact pairs, which are included in whole-model analysis.

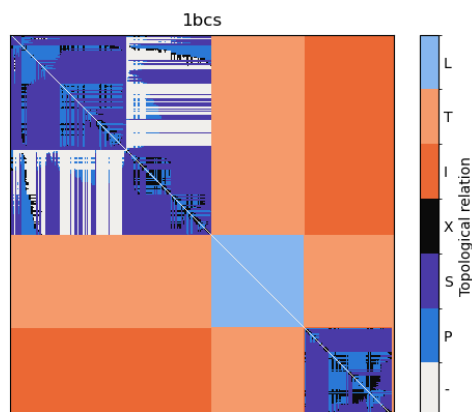


Figure 9: Relations-matrix plot for the same multi-chain object. The legend uses the multi-chain relation types (P, S, X, I, T, L), which appear only for multi-chain proteins. For single-chain inputs the legend reduces to the standard single-chain relation types.

2.2.1 Visualizing Relation Types on the 3D Structure in PyMOL

The **Visualize Circuit Topology by relation type** panel (Figure 7, button row **(I)**) recolours the structure in the PyMOL viewer according to how densely each residue participates in a given relation type. Pressing **Series (S)**, **Parallel (P)** or **Cross (X)** runs the analysis for each chain of the selected object (intra-chain contact pairs only) and then, per chain, calls

```
get_relation_type_vector(mat, index, relation_type, numbering)
color_by_relation(molecule_name, relation_vector, numbering,
    relation_type)
```

`get_relation_type_vector` returns one value per residue that counts how many relations of the selected relation type that residue takes part in. The value is aligned element-for-element with the residue numbering.

`color_by_relation` writes the counts into the B-factor column and colours the residues by them.

Turning counts into colours

Residues that take part in *no* relation of the selected type are white. Every other residue is placed in one of five levels, from pale tint at lowest to full saturation at highest.

The level boundaries are the *quantiles* of the non-zero counts. This accounts for the heavily right-skewed nature of the residue counts. Spread evenly from zero to the maximum, a few outliers would consume the whole scale — in chain A of **1bcs** the Series counts reach 1900, so a residue taking part in 50 relations would sit 2.6% along a white-to-colour ramp and render as white, indistinguishable from a residue taking part in none. Quantile levels guarantee that every shade is used, so low activity reads as low activity rather than as absence.

Trajectories

The buttons act on whichever object is selected in the dropdown. If that object is a trajectory with more than one state, *every* state is coloured: the analysis runs for each frame and the frames are merged into a single multi-state object named `<object>_topo` that you can scrub through with PyMOL's frame controls. Every frame is measured before any frame is coloured, so the whole trajectory shares one scale and one legend — otherwise a residue would change colour between frames merely because that frame's distribution shifted, which is exactly the sort of thing an animation makes look real. For long trajectories this takes a while and uses significant memory. Non-polymer atoms are excluded before the frames are split, so solvent never enters the calculation.

Warnings you may see. Two limitations inherited from ProteinCT are reported at runtime instead of being silently ignored; both are specified in Section 3.2. Contact detection matches on the residue number alone, so where **insertion codes** are in use the atoms of an inserted residue are attributed to the first residue with that number. Neighbour exclusion is applied by position in the residue list rather than by residue number, so residues flanking a **chain break** are treated as sequence neighbours. A representative log for the three-chain structure `1bcs` looks like this:

```
INFO - Chains found: ['A', 'B', 'C']
WARNING - 1bcs_topo_A: 4 residue number(s) occur more than once, i.e. insertion
codes are in use (23, 58, 76, 112).
WARNING - 1bcs_topo_A: 2 chain break(s) in the residue numbering (11->14,
134->137).
INFO - Analysing 1bcs_topo, chain A ...
WARNING - No contacts found for chain C. Skipping visualization...
INFO - Coloured '1bcs_topo' by S relations. '1bcs' keeps its original
B-factors.
```

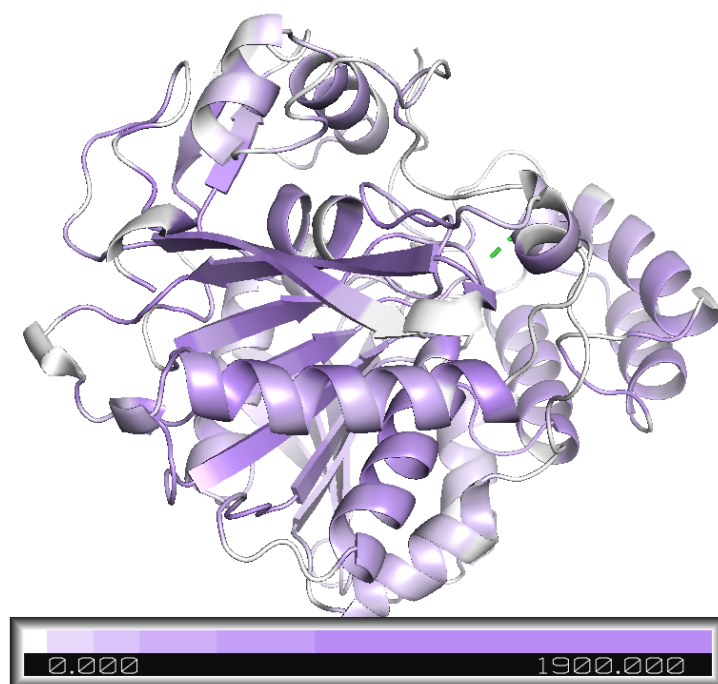


Figure 10: Relation type **Series (S)** visualized for 1bcs. Chain C yielded no contacts and was skipped.

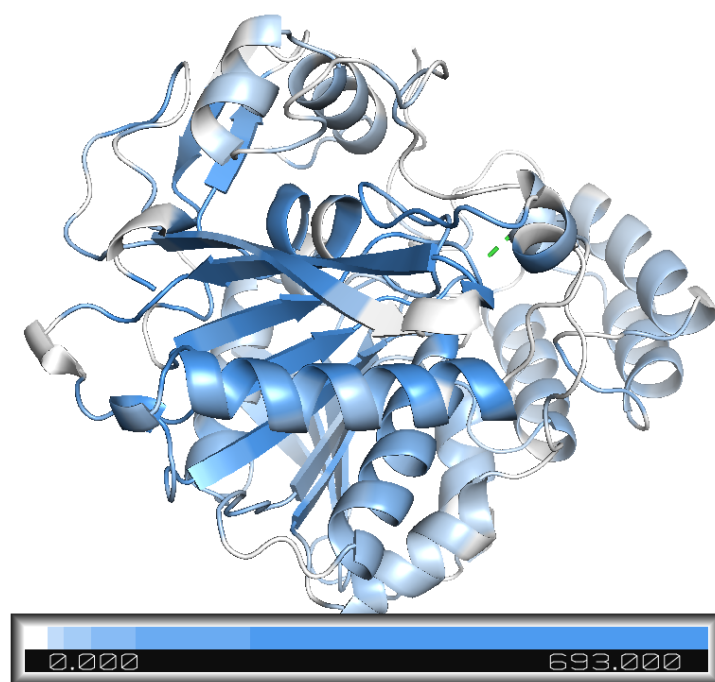


Figure 11: Relation type **Parallel (P)** visualized for 1bcs.



Figure 12: Relation type **Cross (X)** visualized for 1bcs. Comparing the three figures shows which regions are densely involved in each relation type.

2.2.2 CT Folding Score

Ticking **Enable Folding Score (L)** reduces the contact map and relations matrix to a single number per chain: the sum of the mean per-residue participation counts in Parallel, Series and Cross relations, computed by `get_folding_score` from the intra-chain contact pairs of that chain. Larger values indicate a contact network in which the average residue participates in more topological relations. The score builds on the circuit-topology-based fold analysis framework of Akulov *et al.* [3] and gives a quick way to compare conformations without inspecting full plots.

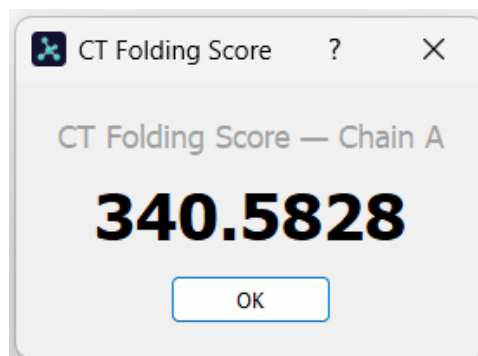


Figure 13: CT Folding Score for protein 1a6n.

2.3 Multi-File Analysis Protocol

The Multi-File Analysis tab batch-processes a set of structures: either a directory of .pdb/.cif files, or a trajectory loaded as a topology .pdb/.cif plus an .xtc trajectory file.

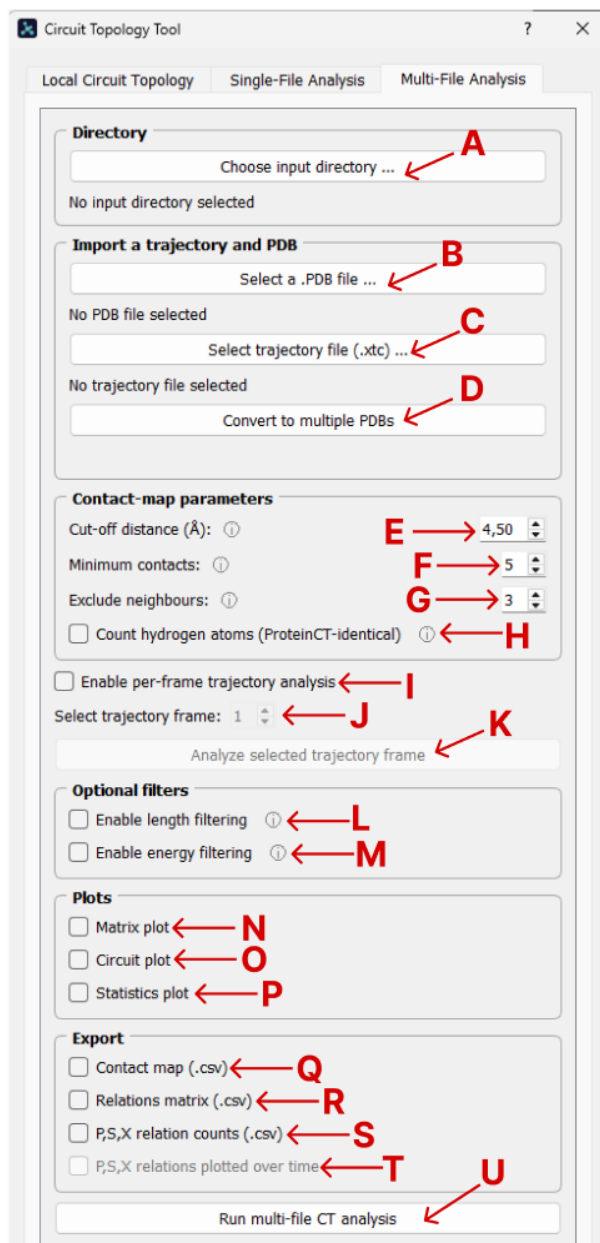


Figure 14: The Multi-File Analysis tab, labelled A–T: directory input (A), trajectory import (B–C), trajectory-to-PDB conversion (D), contact-map parameters (E–G) (the *Count hydrogen atoms* checkbox sits below (G)), per-frame analysis (H–J), optional filters (K) length and (L) energy, plot toggles (M) matrix, (N) circuit, (O) statistics, export checkboxes (P–S), and the run button (T).

Input Modes

Two input modes are supported. Either

- point at a **directory** of structures via **Choose input directory ...** (A), or

- import a topology and trajectory pair: **Select a .PDB file ... (B)** and **Select trajectory file (.xtc) ... (C)**.

With a topology and trajectory loaded, **Convert to multiple PDBs (D)** writes each frame to its own `.pdb` file, turning the trajectory into a directory you can then point at for a regular multi-file run. This is optional: a loaded trajectory can be analysed directly, frame by frame, without exporting anything.

Per-Frame Analysis

Ticking **Enable per-frame trajectory analysis (H)** enables the **Select trajectory frame** spinner (**I**) and the **Analyze selected trajectory frame** button (**J**), which processes only the chosen frame. Both stay greyed out until the checkbox is ticked. As an alternative, the imported trajectory object can be selected on the Single-File Analysis tab and stepped through with PyMOL's own frame controls.

Optional Filters

- **Enable length filtering (K)**: keeps only contacts whose sequence separation matches the chosen comparator — **equality (=)**, **less than (<)** or **greater than (>)** — against a **Distance** threshold. Ticking the box reveals the threshold spinner and the comparator dropdown.
- **Enable energy filtering (L)**: retains only **Attractive / stabilising (+)** or only **Repulsive / destabilising (-)** contacts. Ticking the box reveals the mode dropdown.

Filter caveats

Both filters were designed for single-chain proteins and are skipped automatically when the input contains multiple chains — the plugin reports which file and which filter it skipped. They also have no effect during *per-frame trajectory analysis*, which runs the unfiltered pipeline on the chosen frame. Aggressive length-filter thresholds can produce empty contact maps.

Plots and Exports

Three plot checkboxes are available:

- **Matrix plot (M)** and **Circuit plot (N)** — the same outputs as the Single-File tab, generated for every input file.
- **Statistics plot (O)** — a two-panel figure: the fraction of P + X relations, i.e. the share of contact-pair relations that are Parallel or Cross (non-Series), against distance from the matrix diagonal — the quantity ProteinCT calls the “entangled” fraction, which describes relation types rather than geometric entanglement — beside a pie chart of the Parallel, Series and Cross proportions.

Three export checkboxes write CSV files to the chosen output directory: the residue contact map (**P**), the relations matrix (**Q**), and the P, S and X relation counts per structure (**R**) (**P,S,X relation counts (.csv)**, one row per file or frame). Ticking it

enables the fourth checkbox, **P,S,X relations plotted over time (S)**, which plots the three counts across the input set. That is only meaningful for a genuine trajectory, i.e. when each input file is a consecutive frame of the same structure. As on the other tabs, the output-directory widgets appear as soon as any export is ticked.

Multiple-File Analysis Guide

1. **Load the input:** pick an input directory (**A**) of .pdb/.cif files, or load a topology and trajectory pair (**B–C**).
2. **(Optional) Convert the trajectory to PDBs (D)** if you want the individual frames as standalone files.
3. **Define contact parameters (E–G):** cut-off distance, minimum contacts, exclude neighbours, and whether hydrogen atoms are counted (off by default; Section 3.1). For a molecular dynamics trajectory this setting decides whether the all-atom frames are analysed on their heavy atoms (default) or on every atom.
4. **(Optional) Per-frame analysis (H–I):** tick **Enable per-frame trajectory analysis** and choose the frame index.
5. **(Optional) Apply filters (K–L):** enable length and/or energy filtering, subject to the caveats above.
6. **Pick plots (M–O):** matrix, circuit, statistics.
7. **Pick exports (P–R):** contact map, relations matrix, P, S, X relation counts, and optionally (**S**) for the time series.
8. **Run:** click **Run multi-file CT analysis (T)** to process the whole input set, or **Analyze selected trajectory frame (J)** to process just the chosen frame.

3 Methods specification

This section states what the plugin computes, so that every number, plot and colour it produces can be traced to a definition. The definitions are those of the reference ProteinCT implementation [1]. The plugin departs from ProteinCT in one default, the heavy-atom contact criterion (Section 3.1), and reports two limitations that ProteinCT handles silently (Section 3.2); everything else is inherited unchanged and is checked against ProteinCT by the automated test suite (Section 3.6). Defaults are given for every parameter below; the ranges the GUI accepts are listed in Section 2.1.

3.1 What is analysed

Polymer atoms of the selected object. The analysis sees the polymer atoms of the selected object only (PyMOL’s `polymer` selection). Waters, ions, ligands and all other non-polymer atoms are excluded automatically, and the exclusion is reported on the PyMOL command line. A file chosen on the Local Circuit Topology or Single-File Analysis tab is loaded into PyMOL and then analysed as an object: the polymer atoms of one state are written to a temporary PDB file and parsed with Biopython, and the loaded object is never modified. Files of a Multi-File input folder are parsed directly (PDB or mmCIF), and only the first model of such a file is used. Multi-state objects (NMR ensembles, trajectories) are analysed one state at a time: the current state for the plots, exports, Local Circuit Topology and Folding Score; every state for the relation-type colouring buttons of the Single-File Analysis tab (Section 2.2.1) and on the Multi-File Analysis tab.

Residues removed when the chain is parsed. Three kinds of residues are dropped before contact detection: (i) every residue flagged as a hetero residue (HETATM records), which removes any remaining water, ion or ligand but also *chemically modified residues deposited as heteroatoms*, such as selenomethionine (MSE) or phosphorylated serine, threonine and tyrosine (SEP, TPO, PTR); (ii) nucleic-acid residues (DA, DC, DG, DT, DU, A, C, G, T), so that protein–nucleic-acid complexes are analysed as protein only; and (iii) residues named UNK. A chain left empty by these removals is dropped as well. A removed modified residue leaves a gap in the residue list, and the residues flanking the gap are then treated as sequence neighbours by the exclusion rule below (one of the two situations the plugin warns about).

Alternate conformations. Biopython represents an atom with alternate locations (altloc) as one disordered atom and selects the conformer with the highest occupancy (the first listed, on a tie). Only that conformer’s coordinates enter the contact search; the other conformers are ignored. The plugin neither enumerates nor merges conformers.

Hydrogens. By default the contact criterion uses **heavy atoms only**: atoms whose element is H or D are removed from the atom list before the neighbour search. The element is read from the element column of the file or, when that column is blank (as in many GROMACS exports), inferred from the atom name, so hydrogens are recognised either way. A model with hydrogens therefore yields the same contact pairs as the same model without them, which makes X-ray structures, NMR entries and MD frames comparable. Ticking **Count hydrogen atoms (ProteinCT-identical)** disables the

filter and counts every atom present, exactly as the reference ProteinCT implementation does. On hydrogen-bearing files this yields more contact pairs (for the NMR entry 1AA7 with default parameters: 215 instead of 171), so results obtained in this mode are only comparable between files with the same hydrogen representation. Hydrogen-free files give identical results in both modes.

3.2 Contact pairs

A *contact pair* is a pair of residues that satisfies the following criterion, applied to the atom list defined above.

1. All atom–atom pairs closer than the **cutoff distance** d (default 4.5 Å) are found.
2. Each atom pair is assigned to the pair of residues it belongs to. Residues are identified by their *position* in the residue list of the chain (0, 1, 2, ...), not by their residue number.
3. Atom pairs whose residues lie n or fewer positions apart are discarded (**exclude neighbours**, default $n = 3$; only $|i - j| > n$ is kept).
4. A residue pair is a contact pair when at least m atom pairs remain for it (**number of contacts**, default $m = 5$).

Contact pairs are unordered and each is listed once as (i, j) with $i < j$; the list is sorted by i and then by j . This order defines the rows and columns of the relations matrix and the “distance from the diagonal” of the statistics plot (Section 3.4). A structure needs at least two contact pairs before any relation exists.

Insertion codes and chain breaks. Two situations are handled imprecisely, exactly as in ProteinCT, and are therefore detected and reported rather than silently ignored. (1) Atom pairs are matched to residues by residue number alone, so where insertion codes are in use (two residues sharing a number, e.g. 52 and 52A) the atoms of the inserted residue are attributed to the first residue with that number. (2) Neighbour exclusion works on list positions, so residues on either side of a gap in the numbering (a chain break, an unresolved loop or a removed modified residue) are treated as sequence neighbours. Both are announced with a **WARNING** naming the residue numbers concerned (Section 2.2.1).

Export. **Contact map (.csv)** writes the contact pairs of a chain as a symmetric 0/1 matrix over the residues of that chain, in list order and without header (`<name>_cmap3.csv`). For multi-chain objects this export is written per chain from its intrachain contact pairs.

3.3 Relations and relation types

A *relation* is the arrangement of two contact pairs along the sequence, and its *relation type* depends only on the order of their four endpoints, never on distances or on the global orientation of the chain. The plugin builds the *relations matrix*, with one row and one column per contact pair, whose entry in row (i, j) and column (k, l) holds the code of their relation type; the diagonal is zero.

Single chain. Let (i, j) and (k, l) be two contact pairs with $i < j$ and $k < l$. Table 2 lists the seven relation types, their codes and their definitions. Parallel and concerted parallel are directed: the code says which of the two contact pairs is the inner one, so a matrix entry and its transpose carry mirrored types (P against P^{-1} , CP against CP^{-1}), whereas S, X and CS are symmetric. “Concerted” relations are those in which the two contact pairs share one residue.

Table 2: Relation types for a single chain and their codes in the relations-matrix plot and .csv export. The condition is stated for the entry in row (i, j) , column (k, l) .

Code	Relation type	Condition on the endpoints
1	Series (S)	$j < k$ or $l < i$: the pairs are disjoint and sequential
2	Parallel (P)	$k < i$ and $j < l$: the row pair is nested inside the column pair
3	Inverse parallel (P^{-1})	$i < k$ and $l < j$: the column pair is nested inside the row pair
4	Cross (X)	$i < k < j < l$ or $k < i < l < j$: interleaved endpoints
5	Concerted parallel (CP)	nested with one shared endpoint, row pair inside: $i = k$, $j < l$ or $j = l$, $k < i$
6	Concerted inverse parallel (CP^{-1})	nested with one shared endpoint, column pair inside: $i = k$, $l < j$ or $j = l$, $i < k$
7	Concerted series (CS)	$j = k$ or $l = i$: sequential with the middle residue shared

Whole model. When the selected object has more than one chain, the plugin switches to whole-model analysis automatically and says so on the command line. The atom list covers all chains, the residues of all chains are placed on one concatenated index (so the neighbour-exclusion window also spans a chain boundary), and every contact pair is stored with the chain of each of its residues. Inter-chain contact pairs are therefore included. Relation types are assigned as in Table 3: S, P and X only when both contact pairs lie within one and the same chain; every other relation, i.e. every relation in which at least one of the two contact pairs joins two chains, uses the vocabulary of Heidari *et al.* [4] as adapted in ProteinCT — Independent (I) when the two contact pairs share no chain, Tandem (T) when they share a chain but cannot be joined into a closed path along chains and contacts, and Loop (L) when they can, which for two contact pairs means that both join the same two chains. In whole-model analysis the direction of Parallel relations and the concerted variants are not distinguished: every nested pair of intrachain contact pairs is P and every sequential one is S.

Table 3: Relation types for a whole model (several chains) and their codes. “Intra” denotes a contact pair within one chain, “inter” one that joins two chains. The digit in the labels I2, I3, I4, T2 and T3 is the number of chains the two contact pairs span (a superscript in the notation of Heidari *et al.*).

Code	Relation type	Condition
1	Parallel (P)	both intra, same chain, nested (either direction; a shared endpoint is allowed)
2	Series (S)	both intra, same chain, sequential (a shared middle residue is allowed)
3	Cross (X)	both intra, same chain, interleaved
4	Independent (I)	no chain in common — I2: two intra pairs in different chains; I3: one intra and one inter pair; I4: two inter pairs
5	Tandem (T)	one chain in common but no closed path — T2: an intra pair and an inter pair involving its chain; T3: two inter pairs sharing exactly one chain
6	Loop (L)	two inter pairs joining the same two chains

The labels S, P, X, I, T and L appear in the legend of the relations-matrix plot and in the `.csv` export of the matrix, whose row and column names `[i(A) - j(B)]` give the residue positions and chains of each contact pair. The chain-resolved counts I2, I3, I4, T2, T3 and L are written to the count export of the Multi-File Analysis tab for multi-chain inputs (columns I, T and L of `psxresults.csv`, which replace the P, S and X fractions of the single-chain layout). The two vocabularies are not interchangeable, so single-chain and multi-chain results should not be compared number for number.

Computed per chain. Per-residue colouring (Section 2.2.1), the CT Folding Score (Section 2.2.2), Local Circuit Topology, the length and energy filters and the contact-map export are computed for each chain separately from its intrachain contact pairs, also for multi-chain objects; relations between chains contribute to none of them.

3.4 Derived quantities

Relation-type counts and fractions. The counts P , S and X reported for a structure (pie chart of the statistics plot, **P,S,X relation counts (.csv)**, time series) follow the ProteinCT convention: P counts codes 2, 3, 5 and 6 (Parallel in both directions and the concerted parallel types), S counts codes 1 and 7 (Series and concerted series) and X counts code 4, each over the pairs of contact pairs, i.e. over the upper triangle of the relations matrix. The fractions are these counts divided by their sum.

Fraction of $P + X$ relations against distance from the diagonal. The left panel of the statistics plot is ProteinCT’s “entanglement statistics”. The k -th diagonal of the relations matrix holds the relations between contact pairs k positions apart in the sorted contact list; the plotted value is one minus the share of Series-type entries on that diagonal, i.e. the share of relations that are Parallel or Cross. ProteinCT calls this

quantity the “entangled” fraction; it is a statement about relation types and not about geometric entanglement, which circuit topology does not measure. Two points are fixed by construction, as in ProteinCT: the value for $k = 0$ is 1 (the main diagonal holds no relations) and the value for the last diagonal, $k = N - 1$ for N contact pairs, is 0 (it is not evaluated). For parity the computation is taken verbatim from ProteinCT: on single-chain matrices it counts codes 1 and 7 as Series and on whole-model matrices code 2; a single-chain matrix without any concerted-series relation, which only occurs for very small contact maps, is handled by the whole-model rule, as in the reference implementation.

Local Circuit Topology. The selected residue is chosen by residue number (the first residue with that number, if it occurs more than once). The contact pairs having it as an endpoint are the *target* contact pairs (red in the plots). For the chosen relation type the plugin selects every other contact pair that stands in that relation to at least one target pair (blue). Seen from a target pair, **Parallel (P)** selects contact pairs nested inside it (codes 3 and 6 in its row of the matrix), **Inverse parallel (IP)** contact pairs that enclose it (codes 2 and 5), **Series (S)** sequential ones (codes 1 and 7) and **Cross (X)** interleaved ones (code 4); the concerted variants are thus counted with their plain types. **Run local CT** prints, for every residue position of the chain, the number of contact pairs selected in this way for each of the four types.

Per-residue participation (3D colouring). For one relation type $R \in \{S, P, X\}$, the participation of a contact pair is the number of other contact pairs standing in relation R to it, counting code 1 for S, codes 2 and 3 for P and code 4 for X (the concerted types are not counted here). The value of a residue is the sum of the participations of the contact pairs it belongs to; a residue without contact pairs has the value 0. It measures topological participation, not the number of the residue’s own contacts. Residues with the value 0 are white. The non-zero values are split into at most five levels whose upper bounds are the 20th, 40th, 60th, 80th and 100th percentiles of the non-zero values, rounded up (coinciding bounds are merged), and a residue falls into the first level whose bound is not below its value. The level shades run from 35 % to 100 % of the relation-type colour. The bounds are printed to the PyMOL command line, the colour bar in the viewer is labelled with 0 and the maximum, and the values are written into the B-factor column of the copy `<object>_topo`, so the original object keeps its B-factors. For a trajectory the bounds are computed from all frames together, so that every frame shares one scale.

CT Folding Score. For one chain, the score is the sum of the mean per-residue participation in P, S and X relations as defined in the previous paragraph, the means being taken over all residues of the chain including those with the value 0. It is computed from the intrachain contact pairs of the chain and follows Akulov *et al.* [3].

3.5 Batch and trajectory processing

A folder of structures is processed in natural filename order (digits are compared as numbers, so `frame2` precedes `frame10`). A trajectory is a topology file (`.pdb` or `.cif`) plus an `.xtc` file loaded into PyMOL; its states are processed in order, the polymer atoms of each state being exported to a temporary PDB file and analysed like a single structure, with the same contact-map parameters for every frame. Frames in which no residue

pair satisfies the contact criterion (possible for extended conformations, in particular in heavy-atom mode) are skipped and counted; the final message reports how many were skipped, and skipped frames are absent from the count export and from the time series rather than plotted as zero, so the frame axis of the time series counts analysed frames. Multi-chain frames are analysed at the whole-model level, and the length and energy filters, which are defined for single chains, are then not applied; the plugin names the affected files.

3.6 Validation

The repository's test suite (`tests/`) runs automatically on every commit and, for each release, inside a fresh PyMOL 3.1.6.1 installation on Windows, Linux and macOS. `tests/test_methods_spec.py` is the executable form of this section (hydrogen default and option, parse-time removals, contact criterion, neighbour exclusion by list position, chain-break and insertion-code warnings). `tests/test_upstream_parity.py` compares contact pairs, relations matrices, statistics and circuit plots with the reference ProteinCT code on 1CRN, 1UBQ, 1AKI and 1PNJ, and checks that the **Count hydrogen atoms** mode reproduces ProteinCT exactly on the hydrogen-bearing NMR entry 1AA7. The script `reproduce.pml` regenerates the published lysozyme outputs from a tagged release with one command, also as a Docker capsule; see `README.md` and `REPRODUCING_GUIDE.md` in the repository.

References

- [1] Moes, D., Banijamali, E., Sheikhhassani, V., Scalvini, B., Woodard, J., Mashaghi, A. (2022). *ProteinCT: An implementation of the protein circuit topology framework*. MethodsX 9, 101861. doi:10.1016/j.mex.2022.101861. Source code: https://github.com/circuittopology/circuit_topology.
- [2] Mashaghi, A., Van Wijk, R.J., Tans, S.J. (2014). *Circuit topology of proteins and nucleic acids*. Structure 22(9), 1227–1237. doi:10.1016/j.str.2014.06.015.
- [3] Akulov, V., Jiménez Panizo, A., Estébanez-Perpiñá, E., van Noort, J., Mashaghi, A. (2025). *Phosphorylation regulated conformational diversity and topological dynamics of an intrinsically disordered nuclear receptor*. The Journal of Physical Chemistry B 129(30), 7719–7730. doi:10.1021/acs.jpcb.5c03257.
- [4] Heidari, M., Moes, D., Schullian, O., Scalvini, B., Mashaghi, A. (2022). *A topology framework for macromolecular complexes and condensates*. Nano Research 15(11), 9809–9817. doi:10.1007/s12274-022-4355-x.